Visitor Management System (VMS)

Version 01

Author: Shailesh Singh

Table of Contents

1. Project Overview 3
2. Scope of Work 3
3. System Users 4
4. Functional Requirements 4
5. Non-Functional Requirements 7
6. Technical Specifications 8
7. Data Model Overview 21
8. Audit & Logging Requirements 22
9. Assumptions 22
10. Dependencies 22
11. Risk Considerations 22
12. Acceptance Criteria 23
13. Project Plan – Visitor Management System (VMS) 23

1. Project Overview

1.1 Purpose

The purpose of this project is to design and implement a Visitor Management System (VMS) to digitally manage and track all visitors entering the organization premises.

The system will:

- Record visitor details
- Categorize visitors
- Allow QR-based self-registration
- Notify employees when visitors arrive
- Manage meeting locations
- Handle multiple visitors under a single meeting
- Maintain historical visitor records

1.2 Objectives

- Replace manual visitor logbooks
- Improve security and compliance
- Provide real-time visitor tracking
- Enable automated employee notifications
- Maintain audit trail for all visits

2. Scope of Work

2.1 In-Scope

The system shall:

1. Provide QR-based visitor registration
2. Support multiple visitor categories
3. Allow multiple visitors per meeting
4. Capture complete visitor details
5. Allow visitor to select employee to meet
6. Capture meeting location (Conference Room / Board Room etc.)
7. Send automated notification to host employee
8. Maintain check-in and check-out records
9. Provide admin dashboard

10. Provide reporting and export functionality
11. Maintain visitor history
12. Role-based access control

2.2 Out of Scope

- Biometric integration (unless separately approved)
- Third-party building access control integration
- Payment gateway integration
- Hardware procurement (badge printers, tablets)

3. System Users

User Type	Description
Visitor	Person visiting organization
Reception/Admin	Manages visitor records
Employee/Host	Person being visited
System Administrator	Manages configuration

4. Functional Requirements

4.1 Visitor Registration Module

FR-01: QR Code Access

The system shall generate a QR code displayed at reception.

FR-02: Visitor Form

Upon scanning QR, visitor shall be redirected to registration form.

FR-03: Mandatory Fields

Visitor must fill:

- Visitor Category
- Full Name

- Contact Number
- Email ID
- Organization Name
- Vehicle Number (Optional but configurable mandatory)
- Person to Meet
- Purpose of Visit
- Meeting Location
- Number of Visitors

FR-04: Multiple Visitors Handling

If number of visitors > 1:

- System shall dynamically generate additional visitor name fields
- Each visitor shall be recorded individually
- All visitors linked to same Meeting ID

FR-05: OTP Verification (Optional)

System shall support OTP verification via SMS/email.

4.2 Employee Notification Module

FR-06: Arrival Notification

Upon successful submission:

System shall send notification containing:

- Visitor Name(s)
- Purpose
- Meeting Location
- Contact Number
- Arrival Time

Notification Channels:

- Email
- SMS (Optional)
- Dashboard Alert

FR-07: Employee Confirmation

Employee shall:

- Accept visitor
- Reject visitor
- Request reschedule

4.3 Check-In / Check-Out Module

FR-08: Check-In Recording

System shall record automatic check-in timestamp upon form submission.

FR-09: Check-Out

Reception or employee shall mark check-out manually.

System shall calculate total visit duration.

4.4 Meeting Location Management

FR-10: Location Configuration

Admin shall be able to configure:

- Conference Rooms
- Board Rooms
- Meeting Rooms
- Floor details
- Capacity

4.5 Admin Dashboard

FR-11: Dashboard View

Admin shall view:

- Total visitors today
- Checked-in visitors
- Overstayed visitors
- Upcoming meetings

FR-12: Search & Filter

System shall allow filtering by:

- Date range
- Visitor category
- Employee
- Location

FR-13: Reporting

System shall allow:

- Export to Excel
- Export to PDF
- Visitor history report
- Department-wise visitor report

4.6 User Management

FR-14: Role-Based Access Control

Roles:

- Admin
- Reception
- Employee

Each role shall have defined access permissions.

5. Non-Functional Requirements

5.1 Performance

- System shall support minimum 200 concurrent users
- Page response time < 3 seconds

5.2 Security

- Data encryption at rest
- HTTPS enabled
- Role-based authentication
- Audit logs maintained

- Password encryption

5.3 Availability

- System uptime ≥ 99%
- Daily database backup

5.4 Compliance

- Data retention policy configurable
- Visitor data deletion after X months (configurable)

6. Technical Specifications

6.1 High level Architecture

Frontend Layer

- Web Application (React / Angular / Vue)
- Mobile responsive (QR-based access)
- Optional: Tablet/Kiosk mode at reception

Backend Layer

- REST API (Node.js / .NET Core / Python Django)
- Authentication & Authorization
- Notification service (Email + SMS + Push)

Database Layer

- SQL Server / PostgreSQL / MySQL

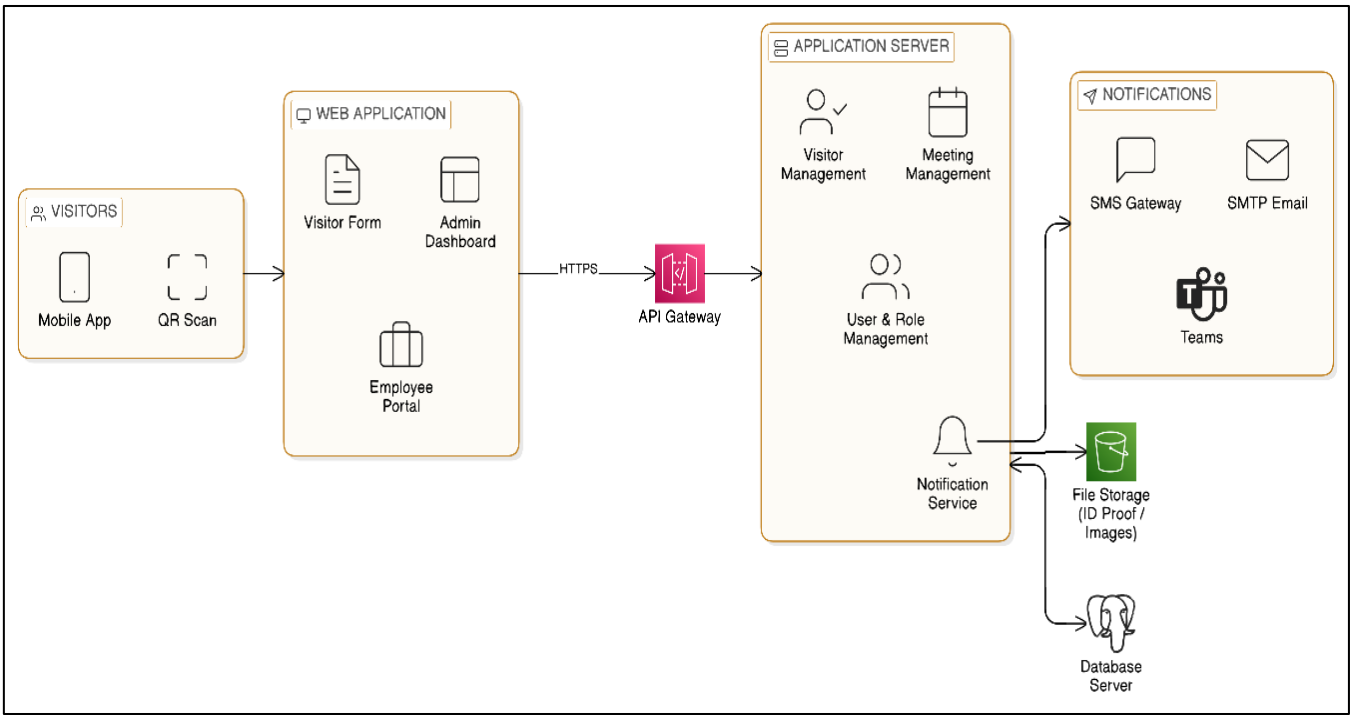
Notification Layer

- Email (SMTP / Microsoft 365)
- SMS Gateway (Twilio / Local provider)
- Optional: Microsoft Teams integration

QR Code Flow

1. Reception generates QR code
2. Visitor scans QR
3. Form opens

4. Visitor fills details
5. System sends notification to employee
6. Entry logged in database



6.2 Architecture Explanation

Presentation Layer (Frontend)

Components:

- Visitor QR Form (Mobile responsive)
- Reception Dashboard
- Admin Panel
- Employee Portal

Responsibilities:

- Capture visitor information
- Display visitor status
- Show notifications
- Manage rooms & employees

Security:

- HTTPS enabled

- Role-based UI access

API Gateway / Authentication Layer

Purpose:

- Central entry point for all requests
- Token validation (JWT)
- Role-based authorization
- Rate limiting (optional)

Application Layer (Business Logic)

Core Services:

Visitor Service

- Create visitor entry
- Handle multiple visitors
- Validate data

Meeting Service

- Link visitors to meeting
- Track check-in/check-out
- Calculate duration

Notification Service

- Send email/SMS alerts
- Log notification status

User Management Service

- Manage Admin / Reception / Employee roles

Database Layer

Relational Database structure:

- One Meeting → Many Visitors
- One Employee → Many Meetings
- One Location → Many Meetings

Key Features:

- Foreign key constraints

- Indexing for performance
- Audit logging
- Backup scheduling

Notification Layer

Triggered when:

- Visitor registers
- Visitor checks out
- Meeting is updated

Channels:

- Email
- SMS
- Internal dashboard alert

Security Architecture

- HTTPS enforced
- Password hashing
- Role-based access control
- Audit trail logging
- OTP verification (optional)
- Data encryption at rest

6.3 Functional Modules

Visitor Registration Module

Fields Required:

- Visitor Category (Vendor / Client / Interview / Delivery / Internal Guest)
- Visitor Name
- Contact Number
- Email
- Organization Name
- Vehicle Number
- Number of Visitors
- Purpose of Meeting

- Person to Meet
- Meeting Location (Board Room / Conference Room)
- Check-in Time
- Check-out Time
- ID Proof (optional upload)

Multiple Visitors Handling

If number of visitors > 1:

System should dynamically generate additional form rows:

- Visitor 1 Name
- Visitor 2 Name
- Visitor 3 Name
  etc.

All linked to same meeting ID.

Employee Notification

When visitor submits form:

System should:

- Send Email notification
- Send SMS (optional)
- Show dashboard notification

Notification Content:

Visitor Name(s)
Purpose
Meeting Room
Contact Details
Arrival Time

Admin Dashboard

Admin should view:

- Today's Visitors
- Upcoming Meetings
- Checked-in Visitors
- Overstayed Visitors
- Visitor history
- Export to Excel

- Filter by date / employee / category

6.4 Database Design (Data Level Information)

Table 1: Visitors	
Column	**Type**
VisitorID (PK)	INT
MeetingID (FK)	INT
FullName	VARCHAR
ContactNumber	VARCHAR
Email	VARCHAR
OrganizationName	VARCHAR
VehicleNumber	VARCHAR
IDProofType	VARCHAR
IDProofNumber	VARCHAR
CreatedDate	DATETIME

Table 2: Meetings	
Column	**Type**
MeetingID (PK)	INT
HostEmployeeID	INT
VisitorCategory	VARCHAR
Purpose	TEXT
LocationID(FK)	INT
CheckInTime	DATETIME
CheckOutTime	DATETIME
Status	VARCHAR (Scheduled / CheckedIn / Completed)

Table 3: Employees	
Column	**Type**
EmployeeID(PK)	INT
FullName	VARCHAR
Email	VARCHAR
PhoneNumber	VARCHAR
Department	VARCHAR

Table 4: Locations	
Column	**Type**
LocationID(PK)	INT
LocationName	VARCHAR

Floor	VARCHAR
Capacity	INT

Table 5: Notifications Log	
Column	**Type**
NotificationID(PK)	INT
MeetingID(FK)	INT
SentTo	VARCHAR
Channel	VARCHAR (Email/SMS)
SentTime	DATETIME
Status	VARCHAR

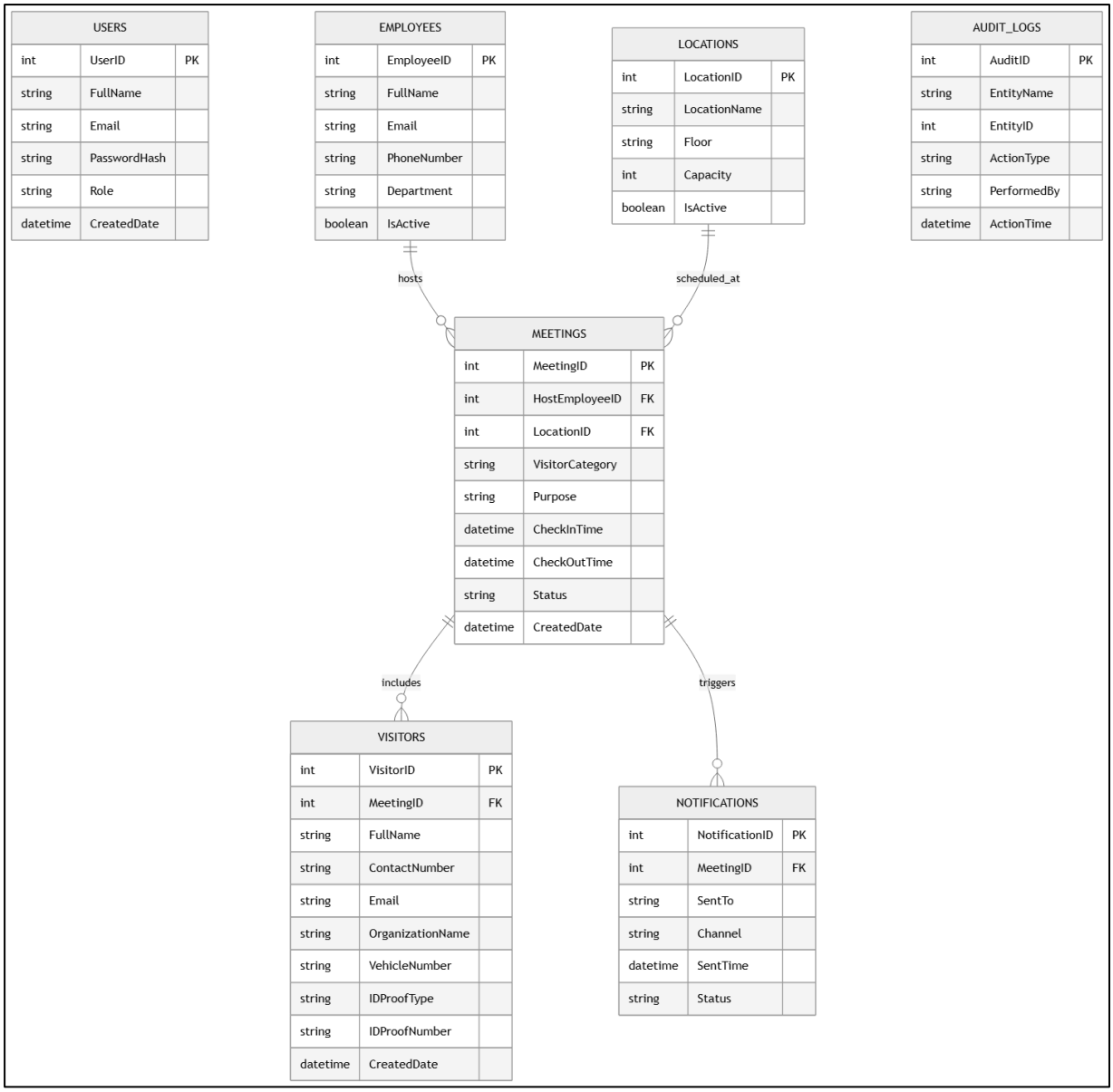
6.5 ER Diagram – Visitor Management System

Main Entities

1. Visitors
2. Meetings
3. Employees
4. Locations
5. Notifications
6. User Roles

Relationships:

- One Meeting → Many Visitors
- One Employee → Many Meetings
- One Location → Many Meetings



6.6 Database VistorManagmentDB

This script includes:

- Primary Keys
- Foreign Keys
- Indexes
- Constraints
- Audit fields
- Default values

```
-- ============================================
-- Create Database
-- ============================================
CREATE DATABASE VisitorManagementDB;
GO

USE VisitorManagementDB;
GO

-- ============================================
-- USERS TABLE (System Login Users)
-- ============================================
CREATE TABLE Users (
    UserID INT IDENTITY(1,1) PRIMARY KEY,
    FullName VARCHAR(150) NOT NULL,
    Email VARCHAR(150) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    Role VARCHAR(50) NOT NULL, -- Admin / Reception / Employee
    IsActive BIT DEFAULT 1,
    CreatedDate DATETIME DEFAULT GETDATE()
);
GO

-- ============================================
-- EMPLOYEES TABLE
-- ============================================
CREATE TABLE Employees (
    EmployeeID INT IDENTITY(1,1) PRIMARY KEY,
    FullName VARCHAR(150) NOT NULL,
    Email VARCHAR(150) NOT NULL UNIQUE,
    PhoneNumber VARCHAR(20),
    Department VARCHAR(100),
```

```
    IsActive BIT DEFAULT 1,
    CreatedDate DATETIME DEFAULT GETDATE()
);
GO

-- ============================================
-- LOCATIONS TABLE
-- ============================================
CREATE TABLE Locations (
    LocationID INT IDENTITY(1,1) PRIMARY KEY,
    LocationName VARCHAR(150) NOT NULL,
    Floor VARCHAR(50),
    Capacity INT,
    IsActive BIT DEFAULT 1,
    CreatedDate DATETIME DEFAULT GETDATE()
);
GO

-- ============================================
-- MEETINGS TABLE
-- ============================================
CREATE TABLE Meetings (
    MeetingID INT IDENTITY(1,1) PRIMARY KEY,
    HostEmployeeID INT NOT NULL,
    LocationID INT NOT NULL,
    VisitorCategory VARCHAR(100) NOT NULL,
    Purpose VARCHAR(500),
    CheckInTime DATETIME DEFAULT GETDATE(),
    CheckOutTime DATETIME NULL,
    Status VARCHAR(50) DEFAULT 'CheckedIn', -- Scheduled / CheckedIn / Completed /
Cancelled
    CreatedDate DATETIME DEFAULT GETDATE(),
```

```
    CONSTRAINT FK_Meeting_Employee
        FOREIGN KEY (HostEmployeeID) REFERENCES Employees(EmployeeID),

    CONSTRAINT FK_Meeting_Location
        FOREIGN KEY (LocationID) REFERENCES Locations(LocationID)
);
GO

-- ============================================
-- VISITORS TABLE
-- ============================================
CREATE TABLE Visitors (
    VisitorID INT IDENTITY(1,1) PRIMARY KEY,
    MeetingID INT NOT NULL,
    FullName VARCHAR(150) NOT NULL,
    ContactNumber VARCHAR(20) NOT NULL,
    Email VARCHAR(150),
    OrganizationName VARCHAR(150),
    VehicleNumber VARCHAR(50),
    IDProofType VARCHAR(50),
    IDProofNumber VARCHAR(100),
    CreatedDate DATETIME DEFAULT GETDATE(),

    CONSTRAINT FK_Visitor_Meeting
        FOREIGN KEY (MeetingID) REFERENCES Meetings(MeetingID)
        ON DELETE CASCADE
);
GO

-- ============================================
-- NOTIFICATIONS TABLE
```

```
-- ==============================================
CREATE TABLE Notifications (
    NotificationID INT IDENTITY(1,1) PRIMARY KEY,
    MeetingID INT NOT NULL,
    SentTo VARCHAR(150) NOT NULL,
    Channel VARCHAR(50) NOT NULL, -- Email / SMS
    SentTime DATETIME DEFAULT GETDATE(),
    Status VARCHAR(50) DEFAULT 'Sent',

    CONSTRAINT FK_Notification_Meeting
        FOREIGN KEY (MeetingID) REFERENCES Meetings(MeetingID)
        ON DELETE CASCADE
);
GO

-- ==============================================
-- AUDIT LOGS TABLE
-- ==============================================
CREATE TABLE AuditLogs (
    AuditID INT IDENTITY(1,1) PRIMARY KEY,
    EntityName VARCHAR(100) NOT NULL,
    EntityID INT NOT NULL,
    ActionType VARCHAR(50) NOT NULL, -- Insert / Update / Delete / Login
    PerformedBy VARCHAR(150),
    ActionTime DATETIME DEFAULT GETDATE()
);
GO

-- ==============================================
-- INDEXES FOR PERFORMANCE
-- ==============================================
```

```
-- Index on Meeting Host
CREATE INDEX IX_Meetings_HostEmployeeID
ON Meetings(HostEmployeeID);

-- Index on Meeting Location
CREATE INDEX IX_Meetings_LocationID
ON Meetings(LocationID);

-- Index on Visitor MeetingID
CREATE INDEX IX_Visitors_MeetingID
ON Visitors(MeetingID);

-- Index on Meeting Status
CREATE INDEX IX_Meetings_Status
ON Meetings(Status);

-- Index on Notification MeetingID
CREATE INDEX IX_Notifications_MeetingID
ON Notifications(MeetingID);
GO
```

6.7 Recommended Technology Stack

Backend:

- .NET Core Web API / Node.js

Frontend:

- React / Angular

Database:

- SQL Server / PostgreSQL

Authentication:

- Azure AD / Internal Auth

Hosting:

- Azure / AWS / On-Prem

Notification:

- SMTP Email
- SMS Gateway API

6.8 API Specifications (Sample)

Create Meeting API

POST /api/meetings

Request JSON:
{
"visitorCategory": "Client",
"purpose": "Project Discussion",
"hostEmployeeId": 102,
"locationId": 3,
"visitors": [
{
"fullName": "John Smith",
"contactNumber": "1234567890",
"email": "john@email.com",
"organizationName": "ABC Ltd",
"vehicleNumber": "CA12345"
}
]
}

Response:
{
"meetingId": 501,
"status": "CheckedIn"
}

7. Data Model Overview

Main Entities

1. Visitors
2. Meetings
3. Employees
4. Locations
5. Notifications
6. User Roles

Relationships:

- One Meeting → Many Visitors
- One Employee → Many Meetings
- One Location → Many Meetings

8. Audit & Logging Requirements

System shall log:

- Visitor submission
- Notification sent
- Check-in
- Check-out
- Record updates
- Login activity

9. Assumptions

- All employees are pre-loaded into system
- Meeting rooms are pre-configured
- QR will be displayed at reception
- Internet connectivity available

10. Dependencies

- SMS provider integration
- Email server configuration
- Hosting environment availability
- Security approval

11. Risk Considerations

- Data privacy risk

- Network downtime
- SMS failure
- Incorrect visitor information

Mitigation:

- OTP validation
- Manual override option
- Admin monitoring dashboard

12. Acceptance Criteria

Project will be considered successful when:

- Visitor can register via QR
- Employee receives notification
- Multiple visitors supported
- Check-in/out recorded
- Admin reports available
- Role-based access working

13. Project Plan – Visitor Management System (VMS)

Project Team Structure		
Role	**Name**	**Responsibility**
Project Manager	Ganesh Poliji	Overall planning, tracking, stakeholder communication
DB Intern 1	Lunathi	Database design, table creation, testing
DB Intern 2	Irene	Data validation, stored procedures, reporting
Frontend Developer (Fresher)	TBD	UI development (Visitor form, dashboard)
Senior DB Support	Senior DB 1	DB architecture review, performance tuning
Senior FE Support	Senior FE 1	UI review, code guidance

High Level Timeline (21 Weeks Plan)					
Phase 1 – Requirements & Design					
Phase 2 – Database Development					
Phase 3 – Backend API Development					
Phase 4 – Frontend Development					
Phase 5 – Integration & Testing					
Phase 6 – Deployment & Go-Live					

Responsibility Matrix (RACI Model)

Activity	PM	DB Interns	FE Fresher	Senior DB	Senior FE
FRS Approval	R				
DB Design	C	R	-	A	A
API Dev	C	C	-	A	-
UI Dev	C	C	R	A	C
Testing	C	R	R	A	A
Deployment	R	C	C	A	A
		Week 3–7		DB Schema Ready	
		Week 6–10		Core APIs Completed	
		Week 9–14		UI Completed	
		Week 13–18		UAT Ready	
		Week 17–21		Production Live	